

Nura AI iOS – System Design Review (Interview Version)

Generated: 2026-05-21 00:53 UTC

1) Why Build This App? What Is It For?

Nura AI is built as a voice-first emotional support companion focused on calm, low-friction conversations. The core goal is to reduce the activation energy for users who are stressed, anxious, or mentally overloaded: users can speak naturally and get immediate guided support.

Unlike productivity assistants, Nura prioritizes emotional pacing, guided micro-interventions (breathing/grounding), and a safe conversational tone. The product target is daily mental wellness support rather than clinical diagnosis.

Business/portfolio angle: this project demonstrates end-to-end AI product execution across iOS UX, backend APIs, realtime-like voice interaction patterns, and production deployment/security hardening.

2) Evolution by Stages (V1 → Vn)

V1 – MVP Scaffold: FastAPI + SQLite backend, SwiftUI iOS shell, auth/onboarding/tab navigation, baseline chat.

V2 – Product UX Baseline: English Calm-style flow, 4-tab IA (Therapist/Meditation/Discover/Profile), onboarding profile capture.

V3 – Voice Foundations: Added STT/TTS APIs and in-app voice responses; initial voice-only mode and orb UI.

V4 – Reliability Hardening: Fixed Xcode project stability, cleaned path/layout issues, prevented startup DB destructive reset, stabilized login/account persistence.

V5 – Hands-Free Conversation Loop: Implemented continuous voice session loop: listening → thinking → speaking → listening with watchdog/self-heal logic.

V6 – Meditation State Machine: Step-by-step meditation guidance with confirmation gating, reduced one-shot multi-step dumps.

V7 – Personalization & Onboarding UX: First-name greeting behavior and post-login welcome splash; stronger profile continuity.

V8 – Public Deployment Readiness: Migrated API base URL to public HTTPS domain (api.zenfrog.tech), reverse proxy/TLS path for non-Tailscale users.

V9 – Voice Experience Tuning: Adaptive VAD thresholds, interruption handling improvements, idle re-engagement prompts, louder TTS normalization/amplification, orb visual quality upgrades.

3) Architecture & Core Components

Client (iOS, SwiftUI): UI state management via AppState, authenticated flow orchestration (Auth → Onboarding → MainTab), TherapistView as the main voice interaction surface, APIClient for typed HTTP requests.

Backend (FastAPI): versioned REST endpoints under /api/v1 for auth, user profile/onboarding, chat, emotion, content, voice (tts/transcribe).

Data Layer: SQLite (MVP) with SQLAlchemy models for users/messages; persistent profile context used for personalization.

AI Layer: OpenAI chat model routing (voice/text-aware), TTS and transcription endpoints, emotion/risk heuristic service for tone pacing.

Voice Session Engine (App side): turn lifecycle, adaptive VAD, idle watchdog, interruption/continue behavior, speaking/listening orchestration.

Deployment: systemd-managed backend service on VPS, HTTPS endpoint through domain-based public API routing.

4) Key Modules (Technical)

- Auth & Identity: email/password auth; onboarding profile as conversational context seed.
- Conversation Service: mode inference (chat/meditation), prompt construction, response generation, risk-aware fallback logic.
- Voice Service: /voice/tts and /voice/transcribe; loudness post-processing pipeline to improve device playback perception.
- Therapist Voice Runtime: adaptive silence thresholding, meaningful transcript gating, idle prompt cycle, auto-exit fail-safe.
- UI/Interaction Layer: text + voice entry paths, keyboard/focus management improvements for smoother form interactions.
- Operations Layer: health checks, systemd restartability, production endpoint migration from private tailnet to public HTTPS.

5) Current Known Challenges

Voice continuity still has edge cases under rapid turn switching (interruptions + idle state transitions + recording restarts).

Realtime parity with ChatGPT/Siri is not yet complete because current implementation is still turn-based over REST, not full streaming realtime transport.

VAD in noisy environments remains sensitive; balancing low latency and false triggers requires iterative tuning and telemetry.

Perceived loudness can vary by device route and OS audio session behavior despite backend normalization.

6) Next Optimization Directions

- Adopt stricter single-writer voice state machine + serialized transition queue.
- Introduce structured voice telemetry (state transitions, timeout causes, transcript quality stats).
- Move toward true streaming/realtime architecture (full-duplex turn-taking) for ChatGPT/Siri-like feel.
- Add configurable voice profiles (normal/loud, calm/energetic pacing) with per-user persistence.
- Improve interruption semantics so user speech always preempts assistant playback reliably.
- Strengthen error recovery contracts (no silent dead states after failed transcript events).

7) Interview Talking Points (Concise)

I built Nura as a voice-first emotional support app, optimized for low-friction, natural conversation rather than command-based UX.

I owned full-stack delivery: iOS client, backend APIs, AI prompt/runtime logic, deployment, and production hardening.

Most complexity was in voice runtime reliability: state machine races, VAD latency/accuracy trade-offs, and interruption behavior.

The current system is production-usable for alpha users, and the next leap is realtime streaming architecture for parity with top-tier assistants.